Input and Output in Hoon

Philip Monk ~wicdev-wisryt
TIon Corporation

Abstract

Urbit's input and output system is structured as explicit state machines, with a small amount of monadic I/O at the top level. This is a deliberate design choice, and we explore the design space.

Contents

Urbit Systems Technical Journal III:2 (2026): 45–58.

Address author correspondence to ~wicdev-wisryt.

1 Introduction

Let's talk about input and output in Urbit. I won't say much about how events and effects are processed by the runtime; it suffices to note that the runtime is a normal Unix program which 1) listens for various events, 2) passes them to Arvo (Urbit OS), and 3) processes the effects produced by Arvo. Instead, I'll focus on how I/O works from the perspective of Hoon programs, like Arvo or apps running on Arvo. But first, let's review I/O paradigms in other languages. [1]

2 The three styles of I/O

I will call these styles imperative I/O, monadic I/O, and state machines (note that this classification refers only to the experience of using them and the program structure they dictate). An imperative programming language can use any of these and a functional programming language can use any of them.

2.1 Imperative I/O

Imperative languages (such as C and Python) perform I/O by calling a function in the middle of whatever they were doing. Anywhere, in any statement, you could have I/O.

In this example and others to follow, the task will be to read a filename from the command line, read a URL out of that file, and fetch the contents at that URL.

```
output = fetch(stripWhitespace(readFile(cliInput())))
```

For readability, intermediate variables would be a good idea, but nothing stops you from nesting I/O this way.

2.2 Monadic I/O

Languages that use explicit monads for I/O (such as Haskell) perform I/O by running I/O behind a bind operation, which

[1] All examples are in pseudocode, since it's useful to understand this before you're comfortable in Hoon. If you want a challenge, try translating them!

I'll notate with ←. Thus, while you can't put I/O *anywhere*, the general structure is a sequence of I/O operations, such as:

```
fileName ← cliInput()
     url ← readFile(fileName)
  output ← fetchUrl(stripWhitespace(url))
```

Simon Peyton Jones' "Tackling the Awkward Squad" (2010) is the canonical description of monadic I/O, and does a great job of explaining how the Haskell community decided to use this as their primary I/O style.

2.3 State machines

Languages that use explicit state machines for I/O (such as Elm) perform I/O by accepting events and producing a list of effects.

```
event :: Initialize
       | CLIInput text
       | FileInput text
       | HTTPResponse body
effect :: CLIInput
        | ReadFile fileName
        | FetchUrl url
        | Print text
main(event) {
  switch event {
    Initialize → [CLIInput ~]
    CLIinput fileName → [ReadFile fileName]
    FileInput url → [FetchUrl stripWhitespace(url)]
    HTTPResponse body → [Print body]
  }
}
```

The effects produced are not functions but data.

It's important to note that this example is chosen to highlight a particular kind of I/O – sequences of events. Further, this example has no persistent state – each I/O operation is a pure function of the output of the previous one.

3 A stateful, loopy example

Let's look at another type of program: a simple HTTP server that you can give text at the CLI, which will respond to all HTTP requests with that text.
Imperative:

```
text = "no text entered yet"
while (true)
  cli = readCli()
  if (cli != Null)
    text = cli
  request = readHttp()
  if (request != Null)
    respond(request,text)
```

Monadic:

```
loop("no text entered yet")
where:
loop :: text → I/O Null
  loop = input ← readInput()
  switch input
    CLIInput newText → loop(newText)
    HTTPRequest request →
      _ ← respond(request, text)
      loop(text)
```

State machine:

```
text = "no text entered yet"
main(text ,event) {
  switch event {
    CLIInput newText → [newText, Null]
    HTTPRequest request → [text, HTTPResponse text]
  }
}
```

Again, note the response in the state machine is data, not a function.

I'm not trying to make a point about which type is better in general. There's a pretty strong argument that in the first example, imperative was the best option, followed by monadic, followed by state machine. There's an equally strong argument

that the second example is most naturally represented by the state machine.

Look at how state is handled in the monadic example. Formally, bind composes functions, so that every time you see a ← there's actually a new function (closure/delimited continuation) being created and stored for when we receive a response. All variables in scope are stored in that closure, so we can conveniently access them later without explicitly storing them. However, the state we preserve over time varies a lot. In this example, it varies between storing `loop`, `[loop input]`, `[loop input newText]`, and `[loop input request]`.

By comparison, the state machine does not store intermediate values, it only stores its explicit state. While this is somewhat less flexible, it puts the permanent state front-and-center.

I'll reiterate that all of these are possible to implement in purely functional languages, and further they can all be implemented in terms of each other. State machines are trivially implementable – it's just a function

```
[state event] → [new-state (list effect)]
```

Monadic I/O can be implemented directly (as in Haskell) or on top of state machines by storing continuations in your state. Imperative I/O is somewhat more difficult but should be possible through algebraic effects.

When inventing a new system, you should consider which of these you intend to support based on the sort of code you expect to see. For example, a scripting language will almost always benefit from imperative (or at least monadic) style.

4 In Urbit

So much for exploring the space of I/O solutions. In terms of deployed code on Earth, the split is about 90% imperative, 5% monadic, and 5% state machine. In Urbit, the split is about 90% state machine and 10% monadic. This is what I intend to explain and defend.

Urbit's OS is called Arvo; it has several "kernel modules" called vanes, and it has userspace applications. Colloquially

these are called "apps," but when speaking about them abstractly, it's helpful to use their more specific name: "agents". Arvo, the vanes, and agents are all structured as explicit state machines.

In addition, there are "threads", which are structured as monadic I/O. Note there's only a superficial resemblance to Unix threads: they are not executed in parallel and they don't share memory, or at least not any more than agents do. They're simply a computation that takes an argument and produces a result, possibly after doing some I/O, structured monadically.

As you can see, all the lowest levels of Arvo are explicit state machines, and only the highest layer supports monadic I/O. To see why this is the case, let's list advantages and disadvantages to each type. Note that, while I'm speaking specifically of agents and threads in the context of Urbit, these are fundamental properties of monadic and state machine I/O.

Agents are:

- Upgradable. Since the state is explicit, you can upgrade an agent in-place by supplying a new state machine and a function from the old state type to the new state type. This is very similar to a database migration.
- Robust. A state machine must accept any input at any time. If it receives unexpected input, it must have had explicit error handling for that input, since it's one of the types in the switch statement. Thus, it's relatively easy to stay in a consistent state.
- Permanent. Since it's upgradable and robust, there's no reason to stop its running.
- Hard to write long sequences of I/O. Long sequences of I/O create many possible states, and you must handle any input in any state. This can result in complex, hard-to-follow code with code paths that are rarely if ever taken.

Arvo's threads are:

- Not upgradable in-place. Since the state is implicit, you can only cleanly upgrade by killing the thread and restarting it, or letting it resolve to a defined "upgradable state"

periodically, usually at the top of a "main loop". In the latter case, then you are in fact structuring it as a state machine, so other advantages of threads are compromised.

- Fragile. If we don't receive expected input, the default behavior is to be stuck waiting for input. You must remember to add any error-handling features you want, and it's easy to end up in an inconsistent state.
- Impermanent. Since a thread can't be upgraded and may get stuck in case of unexpected input, a thread can't be expected to last forever. It must be "rebooted" from time to time.
- Easy to write long sequences of I/O.

Arvo proper and the vanes are permanent pieces of software and, for this reason, state machines are more natural.

I'll note that other kernels, like the Unix kernel, use imperative I/O. The difference is that Unix isn't permanent in itself – it only lasts until you reboot. Since your permanent state is stored externally, you don't need to be upgradable or permanent. With sufficient discipline you can write robust C code, so it makes sense to use imperative style. Even so, a lot of C code is written in a state machine style; even if it doesn't need to be upgradable, it's worth it for the robustness and concurrency advantages.

However, Arvo is a single-level store – all of its state is permanent. This is convenient and eliminates many long-tail bugs, but it also means we need to code for permanence, and that's much easier when you structure code with explicit state.

Further, Arvo and the vanes rarely perform long sequences of I/O. Generally, they do one thing and emit some effects, similar to the HTTP server example above. Sometimes they "pass through" a request. For example, the Eyre vane passes HTTP requests to userspace agents or threads. In this case, there is a sequence of I/O, which in monadic code would be:

```
loop()
where:
  loop = request ← receiveRequest()
```

```
  agent = lookupAgent(request)
  response ← callAgent(agent, request)
  _ ← sendResponse(request, response)
  loop()
```

This is clear code to read, but it leaves open the question of how to respond during the `callAgent()` function. If another request comes in, can we handle it in parallel? If we need to upgrade ourselves before the agent responds, can we make sure to properly handle the response?

The naive solution in monadic code is to say that you can only handle one request at a time, and if an upgrade happens you just drop outstanding requests. You can resolve both of these by introducing a sort of main loop, which is just a way of saying "turn it into a state machine".

In a state machine, the naive solution is:

```
state :: (map request agent)
main(state, event) {
  switch event {
    HTTPRequest request →
      agent = lookupAgent(request)
      [put(state, request, agent),
       CallAgent agent request]
    AgentResponse request response →
      [del(state, request),
       HTTPRespond request response]
  }
}
```

Upgrading this is trivial, since the state (map of requests to outstanding calls to agents) is explicit. It also trivially handles concurrent requests.

I won't claim this is easier to write than a monadic version, but it has the properties we need at this level of the system.

Userspace agents similarly are permanent entities that need to not lose data or get stuck. For this reason they're structured as state machines.

A helpful comparison is that explicit state machines are basically Mealy machines, the kind you might have learned about in an introductory digital design class. This shouldn't

be surprising; a digital circuit must always be consistent because it can't be manually "rebooted", and their input and output is highly formalized. By contrast, a Rube Goldberg machine doesn't have the same constraints; which is why they're a lot more fun!

It's notable also that explicit state machines are considerably more declarative than imperative or monadic I/O. Not everyone thinks that's a good thing, but anyone who defends functional programming should see some advantages to that.

However, userspace sometimes needs to perform long sequences of I/O. Monadic and state machine pseudocode are given in Listing 1 and 2.

Let's look at an example where we want to fetch the front page of a site and the first couple comments on each story. HTTP is unreliable, so on failure we retry five times.

With practice, the state machine version of this can be written correctly. However, 1) it's verbose, 2) the control flow

Listing 1: Monadic I/O example of fetching top stories and comments.

```
topStories ← fetch(topStoriesUrl)
loop(topStories)
where:
loop :: topStories → IO Null
  if topStories is Null
    return Null
  comments ← retryLoop(5,head(topStories))
  otherComments ← loop(tail(topStories))
  append(comments,otherComments)

retryLoop :: [n story] → IO Comments
  comments ← fetchComments(story)
  if comments is HTTPError:
    if n == 0:
      bail
    else:
      retryLoop(n-1, story)
```

Listing 2: State machine example of fetching top stories and comments.

```
state :: Initial
       | FetchingTop
       | FetchingStory comments stories retries
       | Done comments

main(state, event) {
  switch event {
    Initialize →
      assert state == Initial
      [FetchingTop, Fetch topStoriesUrl]
    HttpResponse response →
      switch state {
        FetchingTop →
          [FetchingStory Null stories 5,
           FetchComments head(stories)]
        FetchingStory comments stories retries →
          if response is HTTPError:
            if retries == 0:
              [Initial, Null]
            else:
              [FetchingStory comments stories
                             retries-1,
               FetchComments head(stories)]
          else:
            comments = append(response, comments)
            if tail(stories) is Null:
              [Done comments, Null]
            else:
              [FetchingStory append(comments,response)
                             tail(stories) 5,
               FetchComments head(tail(stories))]
      }
  }
}
```

jumps all over the place, and 3) in practice it's challenging to get exactly right. This is still a small example, and it gets much worse as the length and complexity of the I/O sequence grows.

The monadic version, on the other hand, flows cleanly down the page. We can factor out generic functionality like "retry this request n times" to make the essence of the computation clear.

Suppose you have to upgrade this. In the state machine, you can see exactly which stage of the computation you're in and write specific code to upgrade cleanly.

Threads, as we've discussed, can't be upgraded in-place. Let's be real though: we're downloading comment sections, not regulating a nuclear reactor. If we get a better version of this script, just kill the old one and start the new one from scratch. What're a couple of extra HTTP requests?

This suggests a general division of workloads: code which needs to be permanent, upgradable, and concurrent should generally be a state machine. Code with long or complex sequences of I/O and definite termination should generally be a thread.

These aren't completely mutually exclusive, but it's surprising how often they are.

4.1 Jael: a case study

When you need to do something permanent and upgradable, but you also need to do long sequences of I/O, the main trick is to factor your problem into both an agent and a thread (or several).

As a case study, consider Jael, one of our vanes. This maintains our PKI state, which it downloads over HTTP from an Ethereum node. Downloading from an Ethereum node is a complex sequence that starts with fetching its most recent block number, then fetching all the blocks we haven't seen and scanning for transactions we care about. We also maintain a map of block hashes to block number, and if we see that a block's parent doesn't have the hash we expect, that means a reorganization has occurred, so we must "rewind" one block at a time until we find where the fork happened, then restart going forward.

You don't have to understand this sequence of I/O; it's sufficient to see that it's complex and must be exactly right. We used to do this in a state machine directly in Jael. This data is, of course, permanent and needs to be accessible to the rest of the system, so Jael must be a state machine.

However, we had many small mistakes in this sequence of I/O, some of which only appeared during unusual events, such as when an HTTP request failed at the same time as a reorganization happened.

We're in a bit of a bind, though: Jael has to be a state machine. The solution is to factor out the act of getting updates into a thread while the main PKI state is stored in the state machine in Jael.

This thread has a definite purpose: given the most recent block number we already know about, fetch all the PKI transactions since then. The function signature is:

```
syncEthereum oldBlockNumber →
  I/O [newBlockNumber, newTxs]
```

Jael runs this thread every five minutes and, if it succeeds, then we update our block number and PKI state. If the thread fails, gets stuck, or we need to upgrade it, we just kill the old thread and start a new one. Again, a few extra HTTP requests don't matter.

From Jael's perspective, it looks something like Listing 3. In other words, it's just a single I/O event that encapsulates arbitrarily complex I/O in the thread. A single I/O event is easy enough, and we can maintain all our permanency and robustness guarantees, because we know the thread will have one of three results: success, failure, or it hangs. If we set a timeout, hang becomes failure, so there's only two possible results: success or failure. If we handle both of those correctly, we've handled all the possible I/O errors that could have occurred.

What about upgrading? We've already said we can handle the thread dying for arbitrary reasons, so if an upgrade comes in, we just kill it. We can upgrade Jael's state machine easily since its state is very simple.

Listing 3: Jael's state machine

```
state :: State block txs outstandingThread
main(state,event) {
  switch event {
    Initialize → [State 0 Null Null,
                  StartTimer (now + five minutes)]
    TimerFired →
      effects = If outstandingThread is Null
                then Null
                else [Kill outstandingThread]
      newId = newThreadId()
      [State txs newId,
       append(effects,
              StartThread newId syncEthereum(block),
              StartTimer (now + five minutes))]
    ThreadFinished Failure → [State block txs Null,
                              Null]
    ThreadFinished Success newBlockNumber newTxs →
      [State newBlockNumber append(txs,newTxs) Null,
       Null]
  }
}
```

5 Recap

Let's recap what we've covered:

- There are three styles of I/O: imperative, monadic, and explicit state machines.
- Urbit supports explicit state machines (agents) and monadic I/O (threads).
- Explicit state machines are preferred for permanent, robust, concurrent code.
- Monadic I/O is preferred for short-lived code with complex I/O.
- If a state machine needs to perform complex I/O, it should encapsulate it in a thread so that, from the perspective of the state machine, it's simple I/O.

Hopefully it's fairly clear why Urbit uses explicit state machines in most situations. More than anything else, our goal is to build software that can last forever. Explicit state machines are a crucial tool for that.